

Sang Ah Lee
DATA-260 Section 22
Student ID: 012634544

Configuration:

SID4: 4544
PORT_BASE: 8044
PREFIX: s4544
SEED: 4544
VERIFY_SEED: 264544
DOMAIN_ID: 4 (Open_source package vulnerabilities)

Hardware: Apple M3, 16GB

Model: qwen2.5:3b (switched from qwen3.8b due to hardware issues)

Recent repo state: 950dec2f99d5c01f0aff1809ed72b5260a37f0dc

Link: <https://github.com/sangahlee0/data260-4544>

Link to HW2 Commit # (This commit link does not include the report, because it will be added after downloading this document)

<https://github.com/sangahlee0/data260-4544/tree/436136af6e43623f55f28ee50e047d83ea12d7a1>

Homework 2

Repository Instruction:

- RUN_LOG.txt

```
2745 | --- NODE: Supervisor ---
2746 | {'supervisor': {'turn_count': 1}}
2747 | --- NODE: Planner ---
2748 | {'planner': {'planner_proposal': {'thought': 'The SQL Injection vulnerability is specifically target
2749 | --- NODE: Supervisor ---
2750 | {'supervisor': {'turn_count': 2}}
2751 | --- NODE: Reviewer ---
2752 | {'reviewer': {'reviewer_feedback': {'thought': "SQL Injection flaw in numpy's database interaction,
2753 | --- NODE: Supervisor ---
2754 | {'supervisor': {'turn_count': 3}}
2755 | --- RUN 30 ---
2756 | --- NODE: Supervisor ---
2757 | {'supervisor': {'turn_count': 1}}
2758 | --- NODE: Planner ---
2759 | {'planner': {'planner_proposal': {'thought': 'The SQL Injection vulnerability is exploited through a
2760 | --- NODE: Supervisor ---
2761 | {'supervisor': {'turn_count': 2}}
2762 | --- NODE: Reviewer ---
2763 | {'reviewer': {'reviewer_feedback': {'thought': 'The SQL Injection vulnerability is via a package nam
2764 | --- NODE: Supervisor ---
2765 | {'supervisor': {'turn_count': 3}}
```

- raw/

```
raw
{} adversarial_results.json U
{} ceiling_comparison_results.json
{} schema_experiment_results.json
{} step6_test.json U
```

- METRICS.md

HW02 Metrics						
Method						
Summary of using frozen inputs						
Experiment	Runs	Completion rate	Mean latency (ms)	Median latency (ms)	Minimum (ms)	Maximum (ms)
Schema validation	30	100%	9,172.86	8,877.02	5,190.33	17,487.08
Adversarial input	5	100%	5,906.02	6,102.01	5,223.07	6,507.48
Turn ceiling = 2	20	0%	2,920.50	2,912.14	2,245.90	3,392.20
Turn ceiling = 10	20	100%	11,274.45	10,679.97	5,768.51	18,072.14
Schema Validation Experiment						
Runs	Valid first attempts	Valid-output rate	Mean latency (ms)	Median latency (ms)	Minimum (ms)	Maximum (ms)
30	30	100%	9,172.86	8,877.02	5,190.33	17,487.08

- AI_USE.md

```

### (1) What you used an AI assistant for and what you did yourself;
- I used the AI assistant to understand certain syntax; especially in places where I was unsure why a certain
  necessary. Some specific moments are when I was figuring out the logic between what a planner and reviewer doe
  they were required, and what the difference is between what I had previously, vs a stateful agent graph.
- I searched the internet for HTML examples, and searched up different HTML code to create the website. I also
  myself, only utilizing AI for clarification or for syntax errors.
- I also drafted the md files myself, apart from the METRICS.md where I analyzed my data and formatted the cha
  utilized an assistant to check and make sure the format was more pleasing to see.
- I also utilized AI to figure out how to incorporate Makefile into my project.
- I used AI to create a sample test that I can run as well.
### (2) One AI-produced output that was wrong/unsuitable, or one thing you independently verified;
- An output that I verified was when I was trying to figure out how to run the tests. It gave me a weird idea
  out the results myself, when I knew that there would be a better way to print the results into the RUN_LOG.txt
  clarify it as well.
- I also looked into Pydantic, and looked into the documentation for Experiments and Validations to find the c
  to run the tests.
### (3) How you detected the problem or verified the result;
- I utilized AI to check why I was getting certain error messages, or why instances were not working. I verifi
  results by checking for typos, creating print statements throughout to check, and making sure that what I was
  was meeting the requirements. I always double-checked whatever I changed to make sure it is the best solution
  situation, and flows well with my code.
### (4) What you changed and why it works now;
- I changed the format of my website. I previously did not have CSS, but using the examples given, I utilized
  as reference, and fixing why specific buttons were not working. I had utilized AI to figure out why my website

```

- Report.pdf (saved as LEE_HW2.pdf in the Github repo as well)

This is the current report

- Reproducible run instructions

```

# HW02 Reproducible Run Instructions
Follow these steps to reproduce the experiment and verification script for HW01:

**Needs:** Python 3.11/3.12, Ollama, pydantic, fastapi, uvicorn, Docker, AWS Account (optional for ECS deployment)

1. **Python Version:** Ensure you are using **Python 3.11 or 3.12** (avoid Python 3.13 due to NumPy/LangChain
  compatibility issues).
2. **Ollama Setup:** Ensure Ollama is installed and running locally, and pull the required tool-capable local model:
  ```bash
 ollama serve
 ollama pull qwen2.5:3b
3. **Use a virtual environment:**
  ```bash
  python3 -m venv venv
  source venv/bin/activate
4. Run the Server:
  ```bash
 python3 main.py

```

- verification.json

```

• ((venv)) (base) sangah@MacBook-Air-3 data260-4544 % make verify-hw02
cd code && python3 verify-hw02.py
--- NODE: Supervisor ---
--- NODE: Planner ---
--- NODE: Supervisor ---
--- NODE: Reviewer ---
--- NODE: Supervisor ---
{
 "homework": 2,
 "sid4": 4544,
 "commit_hash": "cb84e09b8448d2099e651605e904b061310dc2aa",
 "model": "qwen2.5:3b",
 "port_base": 8044,
 "seed": 4544,
 "verify_seed": 264544,
 "checks": [
 {
 "check": "LangGraph finishes",
 "passed": true
 },
 {
 "check": "Planner returns exactly 3 tags",
 "passed": true
 },
 {
 "check": "Summary contains at less than or equal to 25 words",
 "passed": true
 },
 {
 "check": "The FastAPI backend responds on PORT_BASE",

```

## Part 1. HTML & CSS

- Style your HW1 entity list/form so it remains usable at 375px width.

```

/*-- for 375px--*/
@media (max-width: 375px) {
 body {
 padding: 10px;
 }
 #feedbackList {
 overflow-x: auto;
 }
}

```

## Open-source package vulnerabilities

Please input/choose the following:

- Package Name
- Vulnerability Name
- Reporter Email
- Content Description
- Severity

Input the open-source package vulnerabilities

Package Name:

e.g. numpy, lodash, etc.

Vulnerability Name:

e.g. Login page security bug

Reporter Email:

Enter your email

Severity:

-- select an option --

Issue Description:

Include a description of the vulnerability issue.

☐ I agree to the terms and conditions.

Report Vulnerability

- Provide visible loading, empty, and error states.

```
<!--Provide visible loading, empty, and error states -->
<p id="loadingState" hidden>Loading</p>
<p id="emptyState"> No vulnerability reports have been submitted.</p>
<p id="errorState" hidden role="alert">There was an issue. Please try again.</p>
```

```
/*--Image for the vulnerability report--*/
#loadingState { color: #666; font-style: italic;}
#emptyState { color: #18682d; font-weight: bold;}
#errorState { color: #cc0000; font-weight: bold;}
```

☐ I agree to the terms and conditions.

Report Vulnerability

No vulnerability reports have been submitted.

## Part 2. FastAPI

1.

```
Start the server
if __name__ == "__main__":
 # Open the browser automatically
 webbrowser.open("http://localhost:8044")
 uvicorn.run(app, host="0.0.0.0", port=8044)
```

# Add record for domain entity

```
@app.post("/api/vulnerabilities", response_model=Vulnerability, status_code=201)
async def create_vulnerability(vulnerability_data: VulnerabilityCreate):
 """Create a new vulnerability - Accepts JSON with vulnerability details"""
 if not vulnerability_data.package_name.strip():
 raise HTTPException(status_code=400, detail="Package name is required")

 # Generate new ID
 new_id = max([v.id for v in vulnerabilities], default=0) + 1
 new_vulnerability = Vulnerability(id=new_id, package_name=vulnerability_data.package_name, vulner
 vulnerabilities.append(new_vulnerability)

 print(f"Created vulnerability: {new_vulnerability}")
 return new_vulnerability
```

# Record is added once user submits data and is redirected showing the updated list

```

// Runs existing validation function, sends the new vulnerability to the API, then res
const submitVulnerability = async (event) => {
 // Prevent the default form submission behavior
 event.preventDefault();
 // Run the existing validation function
 if (!Validation()) return;

 const data = {
 package_name: document.getElementById("PackageName").value,
 vulnerability_name: document.getElementById("VulnerabilityName").value,
 reporter_email: document.getElementById("ReporterEmail").value,
 severity: document.getElementById("Severity").value,
 issue_description: document.getElementById("IssueDescription").value
 };

 try {
 // Send a POST request to the API with the new vulnerability data
 const response = await fetch(API_URL, {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json'
 },
 body: JSON.stringify(data)
 });

 if (!response.ok) throw new Error('Failed to submit vulnerability');
 const newVulnerability = await response.json();
 console.log('New vulnerability submitted:', newVulnerability);

 // Reset the form after successful submission
 document.getElementById("vulnerabilityForm").reset();
 // Refresh the list of vulnerabilities
 loadVulnerabilities();
 }
}

```

## 2. Asd

# Update record with ID 1 to domain-appropriate values for the primary and secondary fields

```
// Update existing record in the list of vulnerabilities
async function updateVulnerability() {
 const packageName = document.getElementById('PackageName').value.trim();
 const vulnerabilityName = document.getElementById('VulnerabilityName').value.trim();
 const emailInput = document.getElementById('ReporterEmail').value.trim();
 const severityInput = document.getElementById('Severity').value;
 const descriptionInput = document.getElementById('IssueDescription').value.trim();

 if (!packageName || !vulnerabilityName) {
 alert('Please enter both Package Name and new Vulnerability Name');
 return;
 }

 try {
 const response = await fetch(`${API_URL}/1`, {
 method: 'PUT',
 headers: {
 'Content-Type': 'application/json',
 },
 body: JSON.stringify({
 package_name: packageName,
 vulnerability_name: vulnerabilityName,
 reporter_email: emailInput,
 severity: severityInput,
 issue_description: descriptionInput
 })
 });

 if (!response.ok) {
 const error = await response.json();
 throw new Error(error.detail || 'Failed to update vulnerability');
 }

 await loadVulnerabilities();
 alert('Vulnerability ID 1 updated successfully');
 } catch (error) {
 console.log(error);
 }
}
```

# Redirect to updated home view and show updated data in list

```
document.getElementById('vulnerabilityForm').addEventListener('submit', submitVulnerability);
// Attach the click event listener to the update button
document.getElementById("updateButton").addEventListener("click", updateVulnerability);
```

```
#updateButton {
 background-color: #2196F3;
}

#updateButton:hover {
 background-color: #43ba67;
}
```

```
<button id="updateButton" type="button">Update Record 1</button>
```

### Input the open-source package vulnerabilities

**Package Name:**

e.g. numpy, lodash, etc.

**Vulnerability Name:**

e.g. Login page security bug

**Reporter Email:**

Enter your email

**Severity:**

-- select an option --



**Issue Description:**

Include a description of the vulnerability issue.

☐ I agree to the terms and conditions.

Report Vulnerability

Update Record 1

3. sd

# Delete record with highest ID

```
// Delete the record with the highest ID in the list of vulnerabilities
async function deleteHighestID() {
 try {
 const response = await fetch(API_URL);
 if (!response.ok) throw new Error('Failed to fetch vulnerabilities');
 const vulnerabilities = await response.json();

 // Check whether list of vulnerabilities is empty before attempting to delete the highest ID
 if (vulnerabilities.length === 0) {
 alert('No vulnerabilities to delete');
 return;
 }

 const highestID = Math.max(...vulnerabilities.map(v => v.id));

 if (!confirm(`Are you sure you want to delete Vulnerability ID ${highestID}?`)) {
 return;
 }

 const deleteResponse = await fetch(`${API_URL}/${highestID}`, {
 method: 'DELETE'
 });

 if (!deleteResponse.ok) {
 const error = await deleteResponse.json();
 throw new Error(error.detail || 'Failed to delete vulnerability');
 }

 console.log(`Deleted Vulnerability ID ${highestID}`);
 await loadVulnerabilities();
 alert(`Vulnerability ID ${highestID} deleted successfully`);
 } catch (error) {
 console.error('Error deleting vulnerability:', error);
 alert('Failed to delete vulnerability: ' + error.message);
 }
}
```

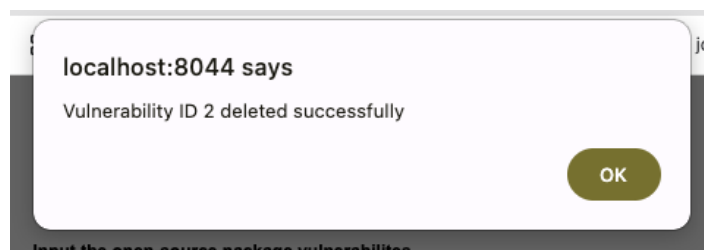
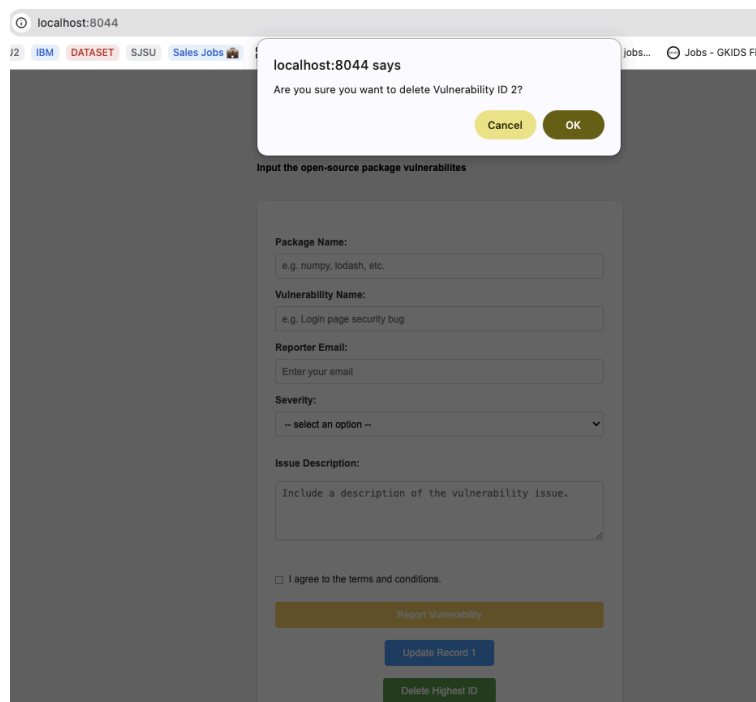
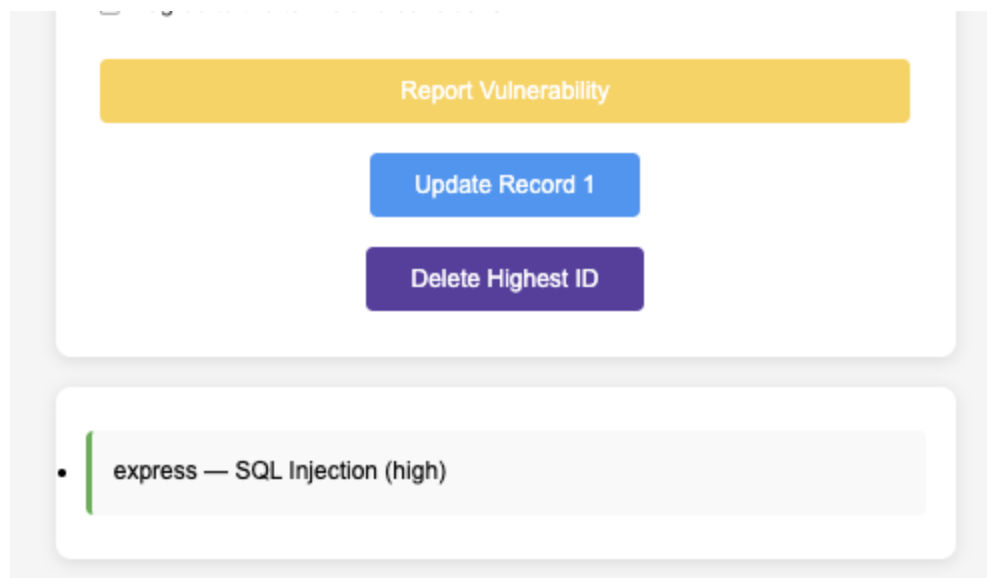
```
#deleteButton {
 background-color: #593d9f;
}

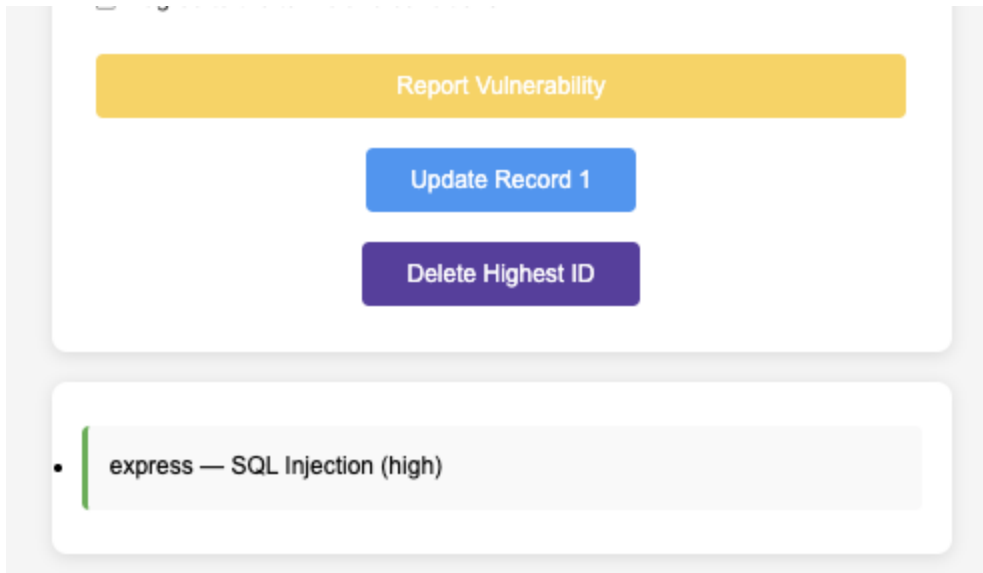
#deleteButton:hover {
 background-color: #1b7b20;
}
```

# After submitting data, redirect to home view and show updated data in list

```
<button id="deleteButton" type="button">Delete Highest ID</button>
```

```
document.getElementById('updateButton').addEventListener('click', updateVulnerability);
// Attach the click event listener to the delete button
document.getElementById("deleteButton").addEventListener("click", deleteHighestID);
```





#### 4. sdf

# Add search functionality and be able to search for record by primary or secondary field

```
const loadVulnerabilities = async (search='') => {
 const loadingEl = document.getElementById("loadingState");
 const emptyEl = document.getElementById("emptyState");
 const errorEl = document.getElementById("errorState");

 loadingEl.hidden = false;
 emptyEl.hidden = true;
 errorEl.hidden = true;

 try {
 const response = await fetch(`${API_URL}?search=${encodeURIComponent(search)}`);
 if (!response.ok) throw new Error('Failed to fetch vulnerabilities');
 const vulnerabilities = await response.json();
 loadingEl.hidden = true;
 displayVulnerabilities(vulnerabilities);
 }
}
```

```
from fastapi import Response

@app.get("/api/vulnerabilities", response_model=List[Vulnerability])
None | None is used to indicate search param can be None or a string
async def get_vulnerabilities(response: Response, search: str | None = None):
 """Get all vulnerabilities - Returns JSON array of vulnerability objects"""
 response.headers["Cache-Control"] = "no-cache, no-store, must-revalidate"
 response.headers["Pragma"] = "no-cache"
 response.headers["Expires"] = "0"

 if search:
 # Filter vulnerabilities based on search query (case-insensitive)
 search = search.lower()
 # Return vulnerabilities where the search term is in package_name or vulnerability_name
 return [v for v in vulnerabilities if search in v.package_name.lower() or search in v.vulnerability_name.lower()]
 return vulnerabilities
```

## Search Vulnerabilities

Search by package or vulnerability name:

Search

Clear Search

- express — SQL Injection (high)
- react — Cross-Site Scripting (medium)

# When the user enters value and searches, the list should update to show only matching records

```
<section id="searchSection">
 <h2>Search Vulnerabilities</h2>
 <input type="text" id="searchInput" placeholder="Search by package or vulnerability name: ">
 <button id="searchButton" type="button">Search</button>
 <button id="clearSearchButton" type="button">Clear Search</button>
</section>
```

```
// Attach the click event listener to the search button
document.getElementById("searchButton").addEventListener("click", async () => {
 const searchValue = document.getElementById("searchInput").value.trim();
 if (!searchValue) {
 alert("Please input a search.");
 return;
 }

 await loadVulnerabilities(searchValue);
});

// Attach the click event listener to the clear search button
document.getElementById("clearSearchButton").addEventListener("click", () => {
 document.getElementById("searchInput").value = '';
 loadVulnerabilities();
});
```

### Search Vulnerabilities

Search

Clear Search

- react — Cross-Site Scripting (medium)

### Part 3. Stateful Agent Graph

#### Step 1:

```

from agents_demo import parse_and_coerce
from model_client import complete
```

#### Step 2:

```
from typing import TypedDict, Dict, Any

TypedDict for the state of the agent, allowing for optional keys
total=False means that all keys are optional, so the state can be partially filled
class AgentState(TypedDict, total=False):
 title: str
 content: str
 email: str
 strict: bool
 task: str
 planner_proposal: Dict[str, Any]
 reviewer_feedback: Dict[str, Any]
 turn_count: int
```

#### Step 3:

# Planner Node

```
def planner_node(state: AgentState) -> Dict[str, Any]:
 print(" --- NODE: Planner --- ")
 # ... (your existing planner logic) ...
 task = state["task"]
 messages = [
 {"role": "system", "content": "Propose exactly 3 distinct, topical tags (prefer multi-word phrases) and a one-line summary for the v"},
 {"role": "user", "content": (f"Task:\n{task}\n"}
 "Return ONLY one JSON object (no code fences, no markdown, no explanations). "
 "Keys: thought (string), message (non-empty, <=60 words, no code), "
 "data.tags (array of exactly 3 topical tags), "
 "data.summary (<=25 words, no ellipses), data.issues (array).\n"
 "Do not add extra text outside JSON.)),]
 # complete is a function that interacts with the LLM to get a response based on the messages provided
 result = complete(messages)
 proposal = parse_and_coerce(result["content"], state["title"], state["content"], state.get("strict", False))
 return {"planner_proposal": proposal}
```

## # Reviewer Node

```
def reviewer_node(state: AgentState) -> Dict[str, Any]:
 print(" --- NODE: Reviewer --- ")
 # ... (your existing reviewer logic) ...
 task = state["task"]

 # proposal is the output from the planner node, which should be passed to the reviewer node and {} if not present
 proposal = state.get("planner_proposal", {})
 messages = [
 {"role": "system", "content": "Validate: tags topical and not generic; summary ≤ 25 words; no code or markdown."},
 {"role": "user", "content": (f"Task:\n{task}\n\n"}
 f"Review the Planner's proposal:\n{proposal}\n\n"
 "Return ONLY one JSON object (no code fences, no markdown, no explanations). "
 "Keys: thought (string), message (non-empty, <=60 words, no code), "
 "data.tags (array of exactly 3 topical tags), "
 "data.summary (<=25 words, no ellipses), data.issues (array).\n"
 "Do not add extra text outside JSON.)),]
 # complete is a function that interacts with the LLM to get a response based on the messages provided
 result = complete(messages)
 feedback = parse_and_coerce(result["content"], state["title"], state["content"], state.get("strict", False))
 return {"reviewer_feedback": feedback}
```

## Step 4:

## # Supervisor Node

```
Supervisor Node modifies the state like incrementing the turn count
def supervisor_node(state: AgentState) -> Dict[str, Any]:
 print(" --- NODE: Supervisor --- ")
 return {"turn_count": state.get("turn_count", 0) + 1}
```

## # Router

```

from __future__ import annotations
from typing import Literal
from state import AgentState

turn_ceiling = 10

def router_logic(state: AgentState) -> Literal["planner", "reviewer", "END"]:
 if state.get("turn_count", 0) >= turn_ceiling:
 return "END"

 if not state.get("planner_proposal"):
 return "planner"
 if not state.get("reviewer_feedback"):
 return "reviewer"

 # If the reviewer has provided feedback and there are issues, go back to the planner for another iteration for the loop. If there are no issues, we can end the
 if state["reviewer_feedback"].get("data", {}).get("issues", []):
 return "planner"
 return "END"

```

## Step 5:

### # Assembling the Graph

```

from __future__ import annotations

from langgraph.graph import StateGraph, END

from state import AgentState
from nodes import planner_node, reviewer_node, supervisor_node
from router import router_logic

def build_workflow():
 workflow = StateGraph(AgentState)

 # Add nodes to the workflow
 workflow.add_node("supervisor", supervisor_node)
 workflow.add_node("planner", planner_node)
 workflow.add_node("reviewer", reviewer_node)

 workflow.set_entry_point("supervisor")

 # Add conditional edges based on the router logic
 workflow.add_conditional_edges("supervisor", router_logic, {
 "planner": "planner",
 "reviewer": "reviewer",
 "END": END
 })

 # Go back to the supervisor after each node to update state (like incrementing turn count)
 workflow.add_edge("planner", "supervisor")
 workflow.add_edge("reviewer", "supervisor")

 return workflow.compile()

```

## Step 6:

# Invoke graph with initial state and use `.stream()` to see output from each step

```
Invoke the graph with the initial state and use .stream() method to see the output of each step
for event in graph.stream(initial_state):
 print(event)
 results.append(event)
```

```
srv update: - prompt 0xb20ce4030: 361 tokens, checkpoints: 0, 10.000 MiB
srv update: - prompt 0xb20ce4970: 218 tokens, checkpoints: 0, 7.667 MiB
srv update: - prompt 0xb20ce4a10: 267 tokens, checkpoints: 0, 9.391 MiB
srv get_availabl: prompt cache update took 8.67 ms
slot launch_slot: id 0 | task -1 | sampler chain: logits -> ?penalties -> ?dry -> ?top-n-sigma -> top-k -> ?typical -> top-p -> ?min-p -> ?xtc -> temp-ext ->
slot launch_slot: id 0 | task -1 | sampler params:
 repeat_last_n = 64, repeat_penalty = 1.000, frequency_penalty = 0.000, presence_penalty = 0.000
 dry_multiplier = 0.000, dry_base = 1.750, dry_allowed_length = 2, dry_penalty_last_n = 64
 top_k = 40, top_p = 0.900, min_p = 0.000, xtc_probability = 0.000, xtc_threshold = 0.100, typical_p = 1.000, top_n_sigma = -1.000, temp = 0.800
 mirostat = 0, mirostat_lr = 0.100, mirostat_ent = 5.000, adaptive_target = -1.000, adaptive_decay = 0.900
slot launch_slot: id 0 | task 2714 | processing task, is_child = 0
slot operator(): id 0 | task 2714 | new prompt, n_ctx_slot = 4096, n_keep = 4, task.n_tokens = 125
slot operator(): id 0 | task 2714 | need to evaluate at least 1 token for each active slot (n_past = 125, task.n_tokens() = 125)
slot operator(): id 0 | task 2714 | n_past was set to 124
slot operator(): id 0 | task 2714 | cached n_tokens = 124, memory_seq_rm [124, end)
slot init_sampler: id 0 | task 2714 | init sampler, took 0.02 ms, tokens: text = 125, total = 125
slot print_timing: id 0 | task 2714 | n_gen = 100, tg = 31.09 t/s, tg_3s = 31.41 t/s
slot print_timing: id 0 | task 2714 | prompt eval time = 33.78 ms / 1 tokens (33.78 ms per token, 29.61 tokens per second)
slot print_timing: id 0 | task 2714 | eval time = 3645.48 ms / 115 tokens (31.98 ms per token, 31.27 tokens per second)
slot print_timing: id 0 | task 2714 | total time = 3679.25 ms / 116 tokens
slot print_timing: id 0 | task 2714 | graphs reused = 2762
slot release: id 0 | task 2714 | stop processing: n_tokens = 239, truncated = 0
srv _update_slots: all slots are idle
```

# Temporarily modify reviewer\_node to always return an issue

```
Extra for Step 6: Running and Testing; temporary modification to always return an issue for testing purposes. Remove this line in production.
feedback["data"]["issues"] = ["Forced test for testing purposes."]
```

```
],
 "summary": "Vulnerable logging to injection attacks.",
 "issues": []
 },
 "reviewer_feedback": {}
},
{
 "supervisor": {
 "turn_count": 2
 },
 "reviewer": {
 "reviewer_feedback": {
 "thought": "The system's logging mechanism is vulnerable to injection attacks.",
 "message": "Injection attacks can corrupt system log entries.",
 "data": {
 "tags": [
 "system-security",
 "logging-vulnerabilities",
 "input-sanitization"
],
 "summary": "Vulnerable logging to injection attacks.",
 "issues": [
 "Forced test for testing purposes."
]
 }
 }
 }
}
```

#### Part 4:

1. Validate Planner output with Pydantic

```
from typing import Annotated

from pydantic import BaseModel, field_validator
from annotated_types import MinLen, MaxLen

class PlannerOutput(BaseModel):
 # Exactly 3 string tags, each 3-30 characters
 tags: Annotated[list[Annotated[str, MinLen(3), MaxLen(30)]], MinLen(3), MaxLen(3)]
 summary: str

 # Summary of at most 25 words
 @field_validator("summary")
 def summary_word_count(cls, summary):
 if len(summary.split()) > 25:
 raise ValueError(f"Summary exceeds 25 words: {len(summary.split())} words")
 return summary
```

2. Feed validation error back to planner and retry within turn ceiling

```
retry_message = ""
if validation_error:
 retry_message = (f"\nPrevious validation error: {validation_error}. \n Fix the error and retry.")

messages = [
 {"role": "system", "content": "Propose exactly 3 distinct, topical tags (prefer multi-word phrases) and a one-line summary for the vulnerability."},
 {"role": "user", "content": (f"Task:\n{task}\n"
 f"Retry message:\n{retry_message}\n"
 f"Return ONLY the 3 tags (as a list) from the model output (do not return anything else)"}]
```

3. Choose fixed input domain and run 30 times

```
{
 "package_name": "numpy",
 "vulnerability_name": "SQL Injection",
 "reporter_email": "test@example.com",
 "severity": "High",
 "issue_description": "User input is not properly sanitized before being used in a SQL query."
}
```

```
Build the compiled LangGraph
with open("../reports/hw02/cases/schema_input.json", "r") as f:
 schema_load = json.load(f)
```

```

retries = planner_attempts - 1

if hit_turn_ceiling:
 outcome = "abandoned at the ceiling"
elif retries == 0:
 outcome = "valid first attempt"
elif retries == 1:
 outcome = "valid after one retry"
else:
 outcome = "valid after two or more retries"

results.append({
 "run": i + 1,
 "outcome": outcome,
})

with open("../reports/hw02/raw/schema_experiment_results.json", "w") as f:
 json.dump(results, f, indent=2)

```

```
> hw02 > raw > {} schema_experiment_results.json > {} 0
```

Click to collapse the range.

```

{
 "run": 1,
 "outcome": "valid first attempt",
 "latency_ms": 8057.989249995444
},
{
 "run": 2,
 "outcome": "valid first attempt",
 "latency_ms": 5778.334292001091
},
{
 "run": 3,
 "outcome": "valid first attempt",
 "latency_ms": 5119.265415996779
},
{
 "run": 4,
 "outcome": "valid first attempt",
 "latency_ms": 5747.324875032064
},
{
 "run": 5,
 "outcome": "valid first attempt",
 "latency_ms": 5997.76112503605
},
{
 "run": 6,
 "outcome": "valid first attempt",
 "latency_ms": 5080.729375011288
},
{

```

```
python3 hw2experiment_run.py 2>61 | tee -a ../reports/hw02/RUN_LOG.txt
added file extensions could lead to arbitrary code execution.', 'data': {'tags': ['insecure-file-upload', 'extension-mismatch', 'code-execution-vulnerability'], 'summary': 'Improper file extension handling leads to code execution.', 'issues': []}}})
--- NODE: Supervisor ---
{'supervisor': {'turn_count': 3}}
--- RUN 30 ---
--- NODE: Supervisor ---
{'supervisor': {'turn_count': 1}}
--- NODE: Planner ---
{'planner': {'planner_proposal': {'thought': 'The vulnerability lies in the improper handling of user input in a complex JSON processing function.', 'message': 'Improper JSON deserialization exposes server to injection attacks.', 'data': {'tags': ['JSONProcessing', 'InputValidation', 'InjectionVulnerability'], 'summary': 'Improper JSON Deserialization.', 'issues': []}}, 'reviewer_feedback': {}, 'validation_error': ''}}
--- NODE: Supervisor ---
{'supervisor': {'turn_count': 2}}
--- NODE: Reviewer ---
{'reviewer': {'reviewer_feedback': {'thought': 'Refine summary to fit constraints and tags.', 'message': 'Improper JSON Deserialization exposes server to injection attacks.', 'data': {'tags': ['InputValidation', 'JSONProcessing', 'InjectionVulnerability'], 'summary': 'Improper JSON Deserialization.', 'issues': []}}}}
--- NODE: Supervisor ---
{'supervisor': {'turn_count': 3}}
(venv) (base) sangah@MacBook-Air-3 code %
```

#### 4. Compare turn ceilings of 2 and 10 over 20 runs

```
"ceiling_2": {
 "turn_ceiling": 2,
 "completion_rate": 0.0,
 "mean_latency_ms": 2920.503739651758,
 "runs": [
 {
 "run": 1,
 "successful completion": false,
 "latency_ms": 3273.381084029097
 },
 {
 "run": 2,
 "successful completion": false,
 "latency_ms": 3233.8199589867145
 },
 {
 "run": 3,
 "successful completion": false,
 "latency_ms": 2748.2305410085246
 },
 {
 "run": 4,
 "successful completion": false,
 "latency_ms": 3069.5710830041207
 },
 {
 "run": 5,
 "successful completion": false,
 "latency_ms": 2707.2712079971097
 },
],
}
```

```

"ceiling_10": {
 "turn_ceiling": 10,
 "completion_rate": 1.0,
 "mean_latency_ms": 11274.448437555111,
 "runs": [
 {
 "run": 1,
 "successful completion": true,
 "latency_ms": 5768.508167006075
 },
 {
 "run": 2,
 "successful completion": true,
 "latency_ms": 12821.337166999001
 },
 {
 "run": 3,
 "successful completion": true,
 "latency_ms": 8115.847749984823
 },
 {
 "run": 4,
 "successful completion": true,
 "latency_ms": 12576.11458399333
 },
 {
 "run": 5,
 "successful completion": true,
 "latency_ms": 16793.888874992263
 },
],
}

```

Based on these two results, I would use a 10 over 20 runs for deployment, as that has a 100% completion rate, while the 2 over 20 runs had a 0% completion rate.

5. Adversarial input and run it 5 times

```

elif args.experiment == 5:
 run_experiment(
 "../reports/hw02/cases/adversarial_input.json",
 5,
 "../reports/hw02/raw/adversarial_results.json"
)

```

```
{
 "package_name": "conflict-package",
 "vulnerability_name": "Multi-Issue Security Report",
 "reporter_email": "test@example.com",
 "severity": "Critical",
 "issue_description": "The report requires preserving all five vulnerability categories separately: SQL injection, authen
}
```

```
[
 {
 "run": 1,
 "outcome": "valid first attempt",
 "latency_ms": 6172.315957956016
 },
 {
 "run": 2,
 "outcome": "valid first attempt",
 "latency_ms": 6507.481792010367
 },
 {
 "run": 3,
 "outcome": "valid first attempt",
 "latency_ms": 6102.010832983069
 },
 {
 "run": 4,
 "outcome": "valid first attempt",
 "latency_ms": 5223.074708017521
 },
 {
 "run": 5,
 "outcome": "valid first attempt",
 "latency_ms": 5525.226166995708
 }
]
```

After testing 3 different adversarial inputs, the observed rate all returned 100% completion rate, and thus a 0% ceiling-hit rate. I had introduced multiple signals, but if the input had caused failures, it may be due to the ambiguity of the descriptions, and a fix would be to be more specific or separate in sentences before starting the process.